Самостійна робота №6

- **Тема:** Приклади поліморфізму в .NET.
- **Обсяг:** 80 хв (2 академічні години)

Мета

Дослідити, як поліморфізм реалізований та використовується в стандартних бібліотеках .NET, а також проаналізувати його переваги на конкретних прикладах.

Завдання

1. **Створити новий консольний проєкт** `IndependentWork5`.
2. **Дослідити поліморфізм через базовий клас `Object`:**
   - Усі класи в C# неявно успадковуються від `System.Object`. Це означає, що вони успадковують такі віртуальні методи, як `ToString()`, `Equals()`, `GetHashCode()`.
   - Створіть 2-3 власні класи (наприклад, `Product`, `Customer`, `Order`).
   - Для кожного класу перевизначте (`override`) метод `ToString()`, щоб він повертав змістовну інформацію про об'єкт.
   - У `Main` створіть об'єкти цих класів і додайте їх до колекції типу `List<object>`.
   - Пройдіться по колекції та виведіть кожен об'єкт у консоль. Проаналізуйте, яка реалізація `ToString()` викликається.
3. **Дослідити поліморфізм через інтерфейс `IEnumerable`:**
   - Багато колекцій у .NET (наприклад, `List<T>`, `Array`) реалізують інтерфейс `IEnumerable` або `IEnumerable<T>`. Це дозволяє перебирати їх елементи в циклі `foreach` поліморфно.
   - Створіть власний клас `MyCustomCollection`, який буде містити внутрішній `List<string>` і реалізовувати `IEnumerable<string>`.
   - Реалізуйте метод `GetEnumerator()` для `MyCustomCollection`.
   - У `Main` створіть екземпляр `MyCustomCollection`, додайте до нього кілька рядків.
   - Продемонструйте, як можна перебирати елементи `MyCustomCollection` за допомогою `foreach`.
4. **Дослідити поліморфізм через абстрактні класи (приклад `Stream`):**
   - У .NET багато класів, що працюють з потоками даних (файли, мережа, пам'ять), успадковуються від абстрактного класу `System.IO.Stream`.
   - У `Main` створіть об'єкти `FileStream` та `MemoryStream`. Для `FileStream` використайте тестовий файл у каталозі вашого проєкту, щоб розв'язок залишався самодостатнім.
   - Створіть метод `ProcessStream(Stream stream)`, який приймає об'єкт типу `Stream` і виконує з ним якісь дії (наприклад, читає кілька байтів).
   - Викличте `ProcessStream` для `FileStream` та `MemoryStream`. Проаналізуйте, як один і той же метод `ProcessStream` працює з різними типами потоків завдяки поліморфізму.
5. **Підготувати звіт (у вигляді коментарів у `Program.cs` або окремого Markdown-файлу):**
   - Опишіть, як поліморфізм проявляється в кожному з трьох досліджених випадків (`Object`, `IEnumerable`, `Stream`).
   - Поясніть, які переваги дає поліморфізм у кожному з цих сценаріїв (наприклад, гнучкість, повторне використання коду).
   - Наведіть фрагменти коду, що демонструють ваші дослідження.

Індивідуальні варіанти

Завдання спільне для всіх студентів: дослідження поліморфізму через `Object`, `IEnumerable` та `Stream` виконується без варіантів.

Методичні вказівки

- Досліджуйте три сценарії послідовно: спочатку `ToString()` через `System.Object`, потім власна реалізація `IEnumerable<string>`, і лише потім робота з `Stream`.
- Для `FileStream` створіть невеликий тестовий файл прямо в коді проєкту (наприклад, через `File.WriteAllText`), щоб розв'язок не залежав від зовнішніх файлів.
- Фіксуйте висновки одразу після кожного сценарію, а не в кінці - так простіше пояснити, чому саме один клієнтський код (`Console.WriteLine`, `foreach`, `ProcessStream`) працює з різними реалізаціями.

Форма звітності

Студент здає посилання на GitHub-репозиторій із проєктом `IndependentWork5`, який компілюється та запускається без помилок, і звіт (у коментарях `Program.cs` або окремим Markdown-файлом) з описом того, як поліморфізм проявляється в кожному з трьох сценаріїв (`Object`, `IEnumerable`, `Stream`), і фрагментами коду, що це підтверджують.

Критерії оцінювання

Оцінка	Критерій
5	Створено проєкт `IndependentWork5`, коректно реалізовано і продемонстровано всі три сценарії: `ToString()` для власних класів через `List<object>`, `MyCustomCollection` з `IEnumerable<string>` і `GetEnumerator()`, `ProcessStream` для `FileStream` і `MemoryStream`, звіт пояснює переваги поліморфізму в кожному сценарії з фрагментами коду.
4	Усе виконано, але є одна несуттєва похибка: наприклад, пояснення для одного сценарію неповне або незначна недбалість у форматуванні коду.
3	Реалізовано і пояснено два з трьох сценаріїв, або звіт формальний без власних висновків.
2	Реалізовано лише один сценарій, або код не компілюється, або звіту фактично немає.
0	Роботу не здано, заняття пропущено без відпрацювання, або роботу виконано іншою особою.

Оцінки 3-5 означають, що роботу здано; 0-2 - роботу треба доопрацювати.

Джерела

- Поліморфізм (Microsoft Docs)
- Базовий клас Object (Microsoft Docs)
- Інтерфейс IEnumerable (Microsoft Docs)
- Абстрактні класи та члени (Microsoft Docs)

Контрольні запитання

1. Як поліморфізм проявляється при роботі з `List<object>`?
2. Що потрібно реалізувати в `MyCustomCollection`, щоб її можна було перебирати через `foreach`?
3. Чому метод `ProcessStream(Stream stream)` є прикладом поліморфної роботи з абстрактним класом?
4. Які переваги дає єдиний інтерфейс роботи для різних типів потоків і колекцій?